语义层即上下文

理想汽车生产级 Agent 的语义工程实践

演讲嘉宾：钱瀚

目录

CONTENT

01 背景与问题

02 方案演进

03 架构与实现

04 案例与收益

05 取舍与展望

01 背景与问题

个人介绍与实践背景

钱瀚

理想汽车大数据平台负责人

曾在百度、京东和字节跳动工作，负责过百万 Core 级计算调度框架和 EB 级数据湖

当前建设方向

大数据平台的云原生化
多模态数据平台
数据语义层建设

平台支撑规模

支撑 180+ 万车
日均处理 10 万亿级车辆信号

一个问题，背后是六类事实

为什么这个数据处理任务失败了？

处理的数据变多了？
计算资源紧张了？
执行过程出异常了？

答案可以生成，结论必须被证明

- 状态
- 日志
- 指标
- 资源
- 事件
- 历史对比

正确对象 × 正确时间窗 × 完整证据链

02 方案演进

第一阶段：Prompt 与通用工具临场组装上下文

Prompt 专家

角色与原则
诊断步骤
工具说明

模型临时选择信息与调用路径

通用底层工具

- K8s / REST
- SLS / Logs
- StarRocks / SQL
- Prometheus / Metrics

上下文足够开放，但缺少统一语义和稳定边界

上下文依赖底层结构

业务问题要临场翻译为
表、字段、关联关系和 SQL

上下文组装不稳定

同一问题可能选择不同的
指标、日志与时间窗

上下文难复用和治理

经验散落在 Prompt 与查询中
变更影响难以评估

第二阶段：专用工具固化场景上下文

把常用对象、查询模板和判断逻辑封装为稳定上下文

PRIMARY

失败诊断、性能分析
直接返回结构化结论

MID

执行阶段、子任务、执行进程
按条件继续下钻

ATOMIC

日志、SQL 与 API
保留长尾探索入口

已知场景的上下文更稳定，覆盖范围却受工具边界限制

一个场景一套上下文

每出现一条新的调查路径
就要新增或改造工具

上下文随工具膨胀

职责重叠、参数相近
Agent 的选择成本反而上升

跨领域难共享

Spark 能力迁移到 Flink
需要重新封装相似上下文

专用工具提升已知场景准确性，未封装问题仍缺少可用上下文

第三阶段：诊断回归控制上下文变更风险

Prompt 与专用工具持续调整，上下文行为也随之变化

01

真实调用录制

保留真实 LLM 与后端返回

02

场景回放

固定输入与外部依赖

03

行为与语义评测

检查工具调用和诊断结论

04

CI 自动回归

Prompt 或工具变化立即验证

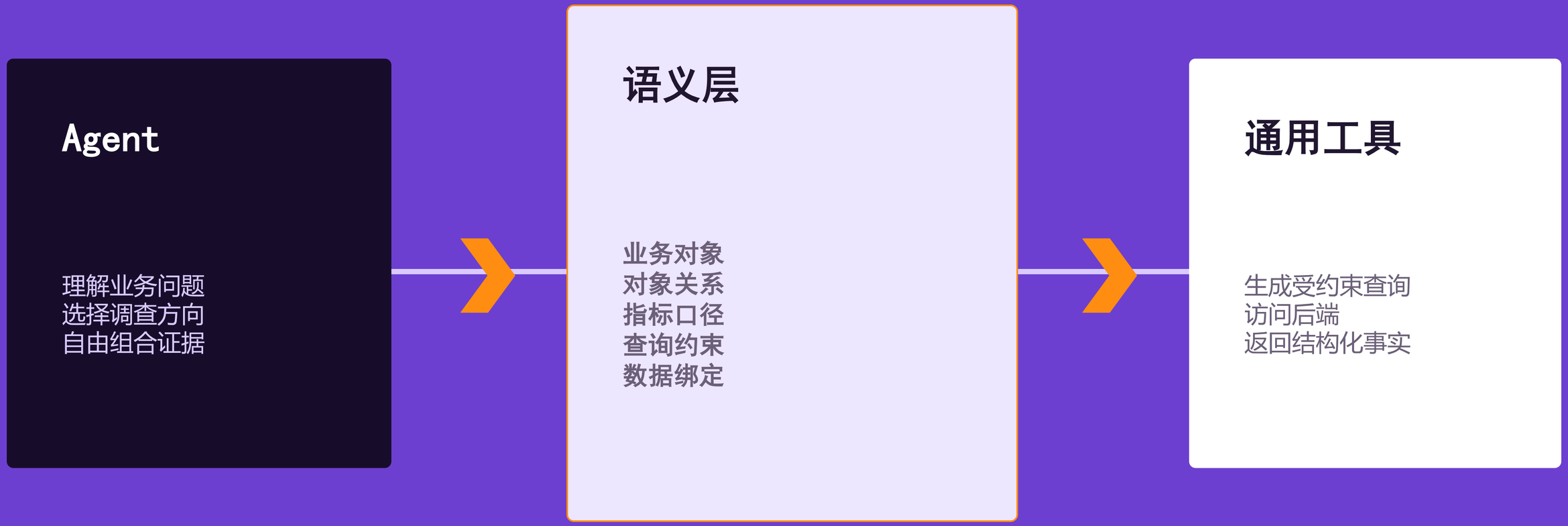
解决的问题	验证上下文变更是否破坏已有场景
留下的问题	回归只能发现波动，无法解决上下文分散和场景扩展成本

早期诊断 Agent 的上下文结构



PRIMARY 固化场景结论，MID 提供对象下钻，ATOMIC 暴露原始事实；上下文仍分散在 Prompt、工具和查询中

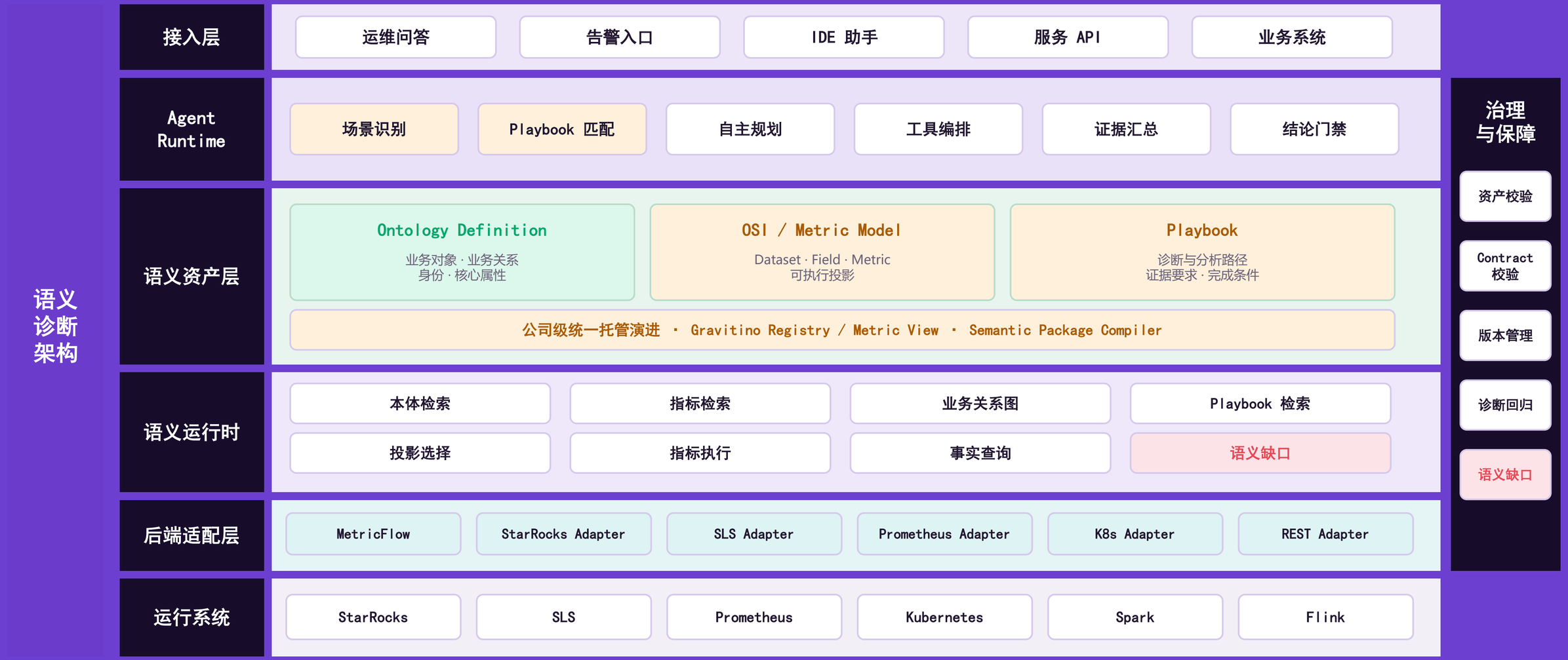
语义层统一支撑准确性与泛化能力



语义层把对象、关系、指标和查询边界组织成可检索、可复用的上下文

03 架构与实现

语义层成为 Agent 的上下文基础设施



本体图表达业务关系，指标图连接可执行证据，Compiler 与 Gravitino 承接统一治理

从业务问题到事实的运行链路



Agent 选择调查路径，语义包同时提供业务关系和可执行指标

业务本体与指标语义分层建设

SEMANTIC PACKAGE

Ontology Definition

业务对象

任务运行（Application）
执行阶段（Stage）
子任务（Task）· 执行进程（Executor）

业务关系

任务包含执行阶段
阶段包含多个并行子任务
子任务运行于执行进程
只表达稳定业务含义，不包含表名和 SQL

OSI / Metric Model

数据集 Dataset

执行阶段表 · 子任务表
spark_stage · spark_task

字段 Field

阶段耗时（毫秒）
stage_duration_ms

可执行投影

物理来源 · 查询约束
StarRocks · Paimon

指标 Metric

子任务耗时倾斜比
task_skew_ratio

本体定义与指标定义分别建模

Ontology Definition

描述业务对象和稳定关系

```
ontology:
- concept:
    name: spark_task
    type: EntityType
    identify_by:
    - dt
    - app_id
    - stage_id
    - task_info_id
  relationships:
  - name: task_to_executor
    roles:
    - concept: spark_executor
    multiplicity: ManyToOne
```

产物：业务关系图

回答对象是什么、彼此如何关联

OSI / Metric Model

描述指标口径和可执行投影

```
metrics:
- name: failed_task_count
  description: 失败 Task 数量
  expression:
    dialects:
    - dialect: ANSI_SQL
      expression: COUNT(CASE WHEN
        task_info_failed = TRUE … END)
  custom_extensions:
  - vendor_name: COMMON
    data: '{"metric_binding": {
      "target_object_type": "spark_task",
      "evidence_role": "failed_task_count"}}'
```

产物：可执行指标图

回答查什么、怎么算、从哪里执行

本体图定义业务世界，指标图把业务对象连接到可执行证据

两类语义资产构建两张图

构建阶段

Ontology Definition

对象、身份、业务关系

Ontology Index

全文与向量索引、关系映射

业务关系图

Application、Stage、Task、Executor

OSI / Metric Model

Dataset、Metric、Projection

Metric Registry

指标依赖、对象绑定、执行能力

可执行指标图

指标口径、对象范围、查询后端

运行时

业务问题

识别对象意图

召回目标对象

Entry Object

1-hop 关系扩展

双向关联对象

对象范围内找指标

目标对象 + 关联对象

Semantic Slice

对象、关系、指标

先确定对象范围，再在范围内找指标，避免把无关指标带入上下文

Agent 先匹配 Playbook，未命中则进入语义层探索

Agent 识别场景

问题意图 · 症状
目标对象

检索并匹配

Diagnosis / Analysis
Playbook

匹配成功 | Playbook 约束调查策略

仍通过语义层选择业务关系、指标和证据

未匹配 | 进入语义层自由探索

从 Object / Relationship / Metric 自主规划

两条路径共享同一语义层和通用工具

01	02	03	04	05
定位对象	选择指标	选择证据	动态补查	结论门禁
对象 / ID / 时间窗	主指标 / 辅助指标	关系路径 / 日志	REST / K8s / 历史对比	**证据 / 置信度 / 边界**

Playbook 提供经验收敛，语义层统一业务关系、指标口径与数据访问

04 案例与收益

一个定时任务，为什么越来越慢？

用户问题：每小时运行一次的数据处理任务，为什么越到晚间越慢？

PLAYBOOK　spark_resource_tuning　调查顺序　历史耗时　执行阶段　子任务分布

01 对比历史耗时

凌晨约半小时
晚间超过 6 小时

任务运行 Application

02 定位慢的阶段

一个执行阶段
就耗时约 4 小时

执行阶段 Stage

03 检查是否有长尾

子任务处理量较均匀
没有明显的个别拖慢

并行子任务 Task

Playbook 指导排查，语义层定位对象与指标：继续追查这个阶段为何慢

根因：反复读取了不需要的数据

继续调查：子任务没有明显长尾，查询到底读取了哪些数据？

PLAYBOOK 调查分支　　SQL 下钻　　语义路径　　查询执行　关联作业　执行阶段与子任务

01 预期读取范围

按小时筛选
只读取需要的时段

查询执行 SQL Execution

02 实际执行方式

小时条件未缩小读取范围
仍读取当天累计数据

执行计划证据

03 执行指标验证

每小时反复读取累计数据
该阶段读取量约 12 TB

阶段指标 Stage

沿对象关系关联查询、执行计划与指标，确认小时过滤未减少实际读取量

语义探索：其他任务是否也在挤占共享资源？

用户追问：其他任务也慢，是不是共同占满了可分配的计算资源？

未命中既有 PLAYBOOK　进入语义层自由探索　探索范围　当前任务　同源任务　共享资源配额

01 发现同类问题

多个任务读取同一数据源
存在相同的查询缺陷

任务运行与查询执行

02 任务跨周期堆叠

上轮未结束，下轮又启动
同一任务最多同时运行 8 份

同一任务的多次运行

03 共享资源紧张

申请的资源接近配额上限

资源队列 resource_queue

超出既有 Playbook 后，Agent 沿语义关系扩展，识别共享资源风险

效果与收益：效率、准确率与治理范围同时提升

10×+

复杂问题排查修复效率

证据收集、根因排查、验证复现到代码修复
从小时/天级缩短到几分钟至十几分钟

97%+

诊断准确率

以受治理语义和真实证据约束诊断过程
减少依赖模型记忆产生的误判

万级/周

系统级风险扫描

覆盖 Spark、Flink 失败与性能退化
聚类形成可治理的候选风险

1 到 N

方法论可跨领域迁移

从单一诊断 Agent 扩展到其他业务领域
复用语义层、Playbook 与通用工具的建设方法

05 取舍与展望

四项边界决定语义层的可维护性

取舍	我们的选择	为什么
固定定义 vs 动态探索	定义对象、关系、指标和查询边界；Agent 动态选择调查路径	兼顾数据口径可靠与复杂追问的灵活性
业务含义 vs 可执行语义	Ontology 定义对象和业务关系；OSI/Metric 定义数据集、指标和执行投影	避免业务概念与表结构、SQL 和后端路由绑死
业务知识 vs 专家 SOP	本体与指标跨场景复用；Playbook 管判断步骤与行动路径	稳定知识和场景策略需要分别治理
语义意图 vs 物理可行性	Physical Spec 与 OSI Projection 管理来源、过滤、时间窗和路由	业务上有关联，不代表当前已有可执行证据

语义资产平台化仍有两个硬问题

01

业务经验如何完成语义抽象

本体、指标和 Playbook
仍需要较多人工梳理

标准化 · 辅助生成 · 数据治理 · 企业数字化建设

02

语义资产如何统一治理

本体、指标和 Playbook 仍以 YAML 分散维护
缺少统一编译、版本与发布链路

Compiler · Registry · 版本审计 · Gravitino 托管

语义资产平台化演进

01	四类资产各归其位	Ontology · Physical Spec · OSI Projection · Playbook
02	Compiler 生成并校验语义包	Registry Rows · Graph Index · Package Lock
03	Gravitino 承接统一治理	Owner · Version · Permission · Lineage
04	Runtime 按任务组装上下文	业务关系图 · 指标图 · Semantic Slice

完整资产统一治理，运行时只装配当前任务需要的上下文

THANKS

演讲嘉宾：钱瀚